

Production Readiness Report

Backend — Pre-Launch Review

Review Date: May 31, 2026 **Reviewer Role:** Principal Backend Architect / Security Engineer / Reliability Engineer / Adversarial QA Engineer / SRE **Verdict:** NOT READY FOR PRODUCTION

This system has critical race conditions in its inventory management core, pervasive authorization bypasses on write paths, and no rate limiting. Deploying to thousands of organizations tomorrow will result in inventory corruption, cross-organization data access, and likely hostile enumeration of sensitive records. The issues below are ordered by severity within each section.

CRITICAL SEVERITY

CRIT-01 — Race Condition: Inventory Double-Spend on Concurrent Offer Acceptance

Affected files: `services/resource-offer.service.ts` — `acceptOffer()`

Exact scenario: Two separate requests arrive simultaneously to accept two different offers that both draw from the same `ResourceLot`. Inside `acceptOffer()`, the transaction reads `offer.resourceLot.availableQuantity`, checks that `offeredQuantity <= availableQuantity`, then decrements. Because Prisma's default PostgreSQL transaction isolation is `READ COMMITTED`, both transactions read the same pre-decrement value, both pass the guard, and both decrement — resulting in `availableQuantity` going negative.

Example attack/failure path:

- `ResourceLot` has `availableQuantity = 100`
- Org A and Org B both submit offers of 80 against the same lot
- Both `acceptOffer` calls land within milliseconds
- Both transactions read `availableQuantity = 100`, both pass `80 <= 100`
- Both commit: `availableQuantity` becomes `-60`

Business impact: Inventory integrity is destroyed. The system has committed to transferring resources it does not have. Financial and reputational loss for organizations relying on resource fulfillment. In humanitarian or supply-chain contexts, this can mean real-world shortages.

Recommended validation test: Fire two concurrent `acceptOffer` calls for offers whose combined quantity exceeds the lot's available quantity. Assert that exactly one succeeds and `availableQuantity` never goes negative after both resolve.

CRIT-02 — Race Condition: TOCTOU in Offer Creation — No Inventory Lock

Affected files: `services/resource-offer.service.ts` — `createResourceOffer()`

Exact scenario: Offer creation reads `lot.availableQuantity` outside any transaction and checks `offeredQuantity > availableQuantity`. This is a classic time-of-check/time-of-use (TOCTOU) gap. Between the read and the offer record insert, another operation (an `acceptOffer`) may decrement `availableQuantity`. Offers are then created against phantom inventory.

Example attack/failure path:

- Lot has `availableQuantity = 10`
- `acceptOffer` runs and decrements to 0 (committed)
- `createResourceOffer` already read `availableQuantity = 10` before the decrement
- Offer for quantity 10 is created and is now valid against a lot with 0 available
- When this offer is later accepted, inventory goes negative (see CRIT-01)

Business impact: Cascading inventory corruption. Invalid offers enter the system that can later be accepted, compounding CRIT-01.

Recommended validation test: Simulate concurrent offer creation and offer acceptance against the same lot at the boundary quantity. Assert no offer is created that exceeds true remaining available quantity.

CRIT-03 — Authorization Bypass: Transfer State Transitions Have No Ownership Check

Affected files: `routes/transfer.routes.ts`, `controllers/transfer.controller.ts`, `services/transfer.service.ts`

Exact scenario: `POST /api/transfers/:id/start` has no `authenticate` middleware. Any unauthenticated caller can advance a transfer from `PENDING` to `IN_TRANSIT`. `POST /api/transfers/:id/complete` and `/:id/cancel` are authenticated but check `requireOrganizationRole` against the organization from `getOrganizationContext` (derived from the request body/params) — there is no check that the acting organization is actually a party to the transfer (either `fromOrganizationId` or `toOrganizationId`).

Example attack/failure path:

- Transfer exists between Org A (supplier) and Org B (recipient)
- Org C's ADMIN authenticates, supplies Org C's `organizationId` in the body
- Org C passes `requireOrganizationRole` because they are ADMIN of Org C
- Org C calls `/complete` — transfer is marked COMPLETED, inventory deducted from Org A's lot, Org B's need marked FULFILLED
- Neither Org A nor Org B authorized this

Business impact: Any organization can complete or cancel any other organization's transfers. Inventory theft via manufactured completions; denial-of-service via mass cancellations.

Recommended validation test: As a user with no relationship to a transfer, call `/complete` and `/cancel` while supplying an arbitrary `organizationId` in the body. Assert 403 is

returned.

CRIT-04 — Authorization Bypass: Resource Lot and Need Creation Require No Authentication

Affected files: `routes/resource-lot.routes.ts`, `routes/resource-need.routes.ts`,
`controllers/resource-lot.controller.ts`, `controllers/resource-need.controller.ts`

Exact scenario: `POST /api/resource-lots` and `POST /api/needs` have no `authenticate` or `requireOrganizationRole` middleware. The `organizationId` is taken directly from the request body with no validation that the caller is a member of that organization.

Example attack/failure path:

- Attacker calls `POST /api/resource-lots` with any valid `organizationId` and `resourceId`
- A ResourceLot is created under a victim organization with arbitrary quantity
- Attacker (or a colluding party) then creates offers against that lot
- Alternatively, spam-creates hundreds of ResourceNeeds under victim organizations, flooding their dashboards

Business impact: Any anonymous user can create inventory records and resource needs under any organization. Cross-organization data injection. Enables downstream inventory manipulation.

Recommended validation test: Without any auth token, POST to `/api/resource-lots` with a valid `organizationId`. Assert 401. With a valid token for a user NOT in that organization, assert 403.

CRIT-05 — Authorization Bypass: Resource Offer Creation Requires No Authentication

Affected files: `routes/resource-offer.routes.ts`, `controllers/resource-offer.controller.ts`

Exact scenario: `POST /api/offers` has no authentication middleware. `offeringOrganizationId` is taken from the request body. Any anonymous caller can create offers claiming to be from any organization, using any ResourceLot.

Example attack/failure path:

- Attacker enumerates a valid `resourceLotId` and `needId` (both accessible via unauthenticated GET endpoints)
- Attacker creates an offer for the full quantity of the lot, claiming to be from the owning org
- Victim org accepts the offer — inventory is reserved from a lot the attacker has no rights over
- This can be used to lock out legitimate offers or deplete competitor inventory

Business impact: Unauthorized reservation of another organization's inventory. Enables denial-of-resource attacks between organizations.

Recommended validation test: Without authentication, POST to `/api/offers` with valid need and lot IDs. Assert 401.

CRIT-06 — Authorization Bypass: Partnership Endpoints Entirely Unprotected

Affected files: `routes/partnership.routes.ts`, `controllers/partnership.controller.ts`, `services/partnership.service.ts`

Exact scenario: Both `POST /partnerships` and `PATCH /partnerships/:id` have no authentication or authorization middleware whatsoever. Any anonymous caller can create a partnership between any two organizations, or update an existing partnership's status (e.g., change `PENDING` to `ACCEPTED`).

Example attack/failure path:

- Attacker calls `PATCH /partnerships/:id` with `{ "status": "ACCEPTED" }` for any partnership ID
- A pending partnership between two organizations is accepted without either party's knowledge or consent
- Partnerships likely gate access to resource sharing features — this bypasses the entire trust model

Business impact: Complete bypass of the inter-organization trust model. Attackers can force-accept partnerships, potentially enabling resource access or data visibility that the target organizations never agreed to.

Recommended validation test: Without authentication, PATCH any partnership to `ACCEPTED`. Assert 401.

CRIT-07 — Authorization Bypass: `acceptOffer` Accepts Client-Supplied `userId` for Approval

Affected files: `controllers/resource-offer.controller.ts` — `acceptOfferController()`

Exact scenario: The `acceptOfferController` reads `userId` from `req.body` and passes it as `approvedById` to `acceptOffer()`. The authenticated user (`req.user.id`) is ignored for this purpose. A caller can impersonate any user as the approver of a transfer.

Example attack/failure path:

- Authenticated user with id `user-attacker` calls `POST /api/offers/:id/accept`
- Body contains `{ "organizationId": "org-victim", "userId": "user-ceo" }`
- Transfer is created with `approvedById = "user-ceo"` — the CEO is recorded as having approved a transfer they never saw
- Audit log records the attacker's token user for the audit event, but the transfer record itself points to the impersonated user

Business impact: Audit trail falsification. Impersonation of senior personnel in financial/resource transfer approvals. Compliance and legal liability.

Recommended validation test: Accept an offer while supplying a different `userId` in the body. Assert the created transfer's `approvedById` equals `req.user.id`, not the body value.

CRIT-08 — Missing Pessimistic Lock in `acceptOffer` Transaction

Affected files: `services/resource-offer.service.ts` — `acceptOffer()`

Exact scenario: The `acceptOffer` transaction uses `findUnique` to read the `ResourceLot` and then performs an `update`. Under `READ COMMITTED` isolation (Prisma/PostgreSQL default), two concurrent transactions can both read the same `availableQuantity`, both pass the `offeredQuantity <= availableQuantity` guard, and both successfully decrement — resulting in negative inventory. There is no `SELECT ... FOR UPDATE` or equivalent pessimistic lock on the `ResourceLot` row.

Example attack/failure path: Identical to CRIT-01 but specifically within the `acceptOffer` transaction rather than across offer creation. Even if CRIT-01 is fixed, concurrent `acceptOffer` calls on different offers for the same lot can race past the guard.

Business impact: Same as CRIT-01 — inventory corruption at scale.

Recommended validation test: Fire 10 concurrent `acceptOffer` calls for offers covering the full lot quantity. Assert total accepted quantity never exceeds available quantity.

HIGH SEVERITY

HIGH-01 — Membership Status Never Checked in Authorization

Affected files: `services/authorization.service.ts` — `getMembership()`, `requireRole()`

Exact scenario: `getMembership` fetches by `userId_organizationId` unique key and returns the record if it exists, regardless of the membership's `status` field. `requireRole` calls `requireMembership` which calls `getMembership`. A membership with `status = "PENDING"` or a hypothetically `"SUSPENDED"` status grants full role-based access as if it were active.

Example attack/failure path:

- User is invited to an organization, membership created with `status = "PENDING"`
- Before the admin approves, the user can already call any protected endpoint for that organization
- An admin creates a membership for an ex-employee but intends to approve it later — ex-employee has immediate access

Business impact: Premature access grants. If membership approval is a business process, it is entirely bypassed.

Recommended validation test: Create a membership with `status = "PENDING"`. Attempt to call a role-protected endpoint as that user. Assert 403.

HIGH-02 — `cancelResourceNeed` Does Not Handle ACCEPTED Offers or Active Transfers

Affected files: `services/resource-need.service.ts` — `cancelResourceNeed()`

Exact scenario: When a need is cancelled, the service withdraws all `PENDING` offers. However, it does not handle offers in `ACCEPTED` status, which have already created a `Transfer` record and have already decremented `availableQuantity` from a `ResourceLot`. Cancelling the need leaves orphaned `ACCEPTED` offers, active transfers in `PENDING`/`IN_TRANSIT` state, and inventory permanently reserved with no path to restoration.

Example attack/failure path:

- Need has an `ACCEPTED` offer with an `IN_TRANSIT` transfer
- Admin calls cancel on the need — need becomes `CANCELLED`
- Transfer is still `IN_TRANSIT` with no way to be completed (its need is cancelled)
- `ResourceLot` `availableQuantity` remains decremented forever
- Inventory is permanently locked

Business impact: Permanent inventory leak. In resource-constrained environments, this silently destroys supply capacity.

Recommended validation test: Cancel a need that has an `ACCEPTED` offer and an `IN_TRANSIT` transfer. Assert either: the cancellation is rejected with a clear error, OR the transfer is auto-cancelled and inventory is restored.

HIGH-03 — `updateTransferStatus` Completion Does Not Re-Read Committed Transfers

Affected files: `services/transfer.service.ts` — `updateTransferStatus()`

Exact scenario: When completing a transfer, the need fulfillment calculation fetches the need including `transfers`, then filters for `status === 'COMPLETED' || t.id === transferId`. The current transfer being completed is still `PENDING` in the database at the time of the `findUnique` call (within the same transaction). The filter uses `|| t.id === transferId` as a workaround. However, if another transfer for the same need was completed concurrently between the `findUnique` and the status update, both transactions will compute fulfillment based on stale data and may both set the need to `FULFILLED` — or worse, may both fail to set it.

Example attack/failure path:

- Need for quantity 100; two transfers of 50 each are in progress
- Both `updateTransferStatus(..., 'COMPLETED')` calls run concurrently
- Both transactions read the need with 0 previously completed transfers
- Both compute `totalFulfilled = 50` and set need status to `PARTIALLY_FULFILLED`
- Need never reaches `FULFILLED` despite both transfers completing

Business impact: Need status permanently stuck at `PARTIALLY_FULFILLED` even when fully satisfied. Operational dashboards show incorrect fulfillment state; organizations continue seeking resources they already have.

Recommended validation test: Concurrently complete two transfers that together satisfy a need. Assert the need ends in `FULFILLED` status.

HIGH-04 — `updateResourceLot` Has TOCTOU: Reads Then Updates Without Transaction

Affected files: `services/resource-lot.service.ts` — `updateResourceLot()`

Exact scenario: `updateResourceLot` calls `getLotById(id)` outside a transaction, validates constraints (`available ≤ quantity`, `available ≥ 0`), then calls `prisma.resourceLot.update()` as a separate operation. Between the read and the write, another operation (e.g., `acceptOffer`) may have changed `availableQuantity`. The update then writes potentially inconsistent values.

Example attack/failure path:

- Lot: `quantity = 100`, `availableQuantity = 50`
- Admin calls `updateResourceLot` to set `quantity = 60` — reads `availableQuantity = 50`, passes `50 ≤ 60` guard
- Concurrently, `acceptOffer` decrements `availableQuantity` to 10
- Admin's update writes `quantity = 60` with the new data field — `availableQuantity` is now 10, which is valid, but the admin was computing against 50

Business impact: Inventory invariants can be broken silently under concurrent load.

Recommended validation test: Concurrently update a lot's quantity and accept an offer against it. Assert inventory invariants hold after both complete.

HIGH-05 — No Rate Limiting on Auth Endpoints

Affected files: `app.ts`, `routes/auth.routes.ts`

Exact scenario: There is no rate limiting, IP throttling, or account lockout on `POST /api/auth/login` or `POST /api/auth/register`. An attacker can perform unlimited credential stuffing or brute-force attacks against any known email address.

Example attack/failure path:

- Attacker obtains a list of organization member emails via the unauthenticated `GET /memberships/org/:id` endpoint
- Launches automated password spray against `/api/auth/login` — thousands of attempts per minute
- No lockout, no CAPTCHA, no rate limit — eventually succeeds

Business impact: Account takeover at scale. Compromised accounts with OWNER/ADMIN roles allow full organizational data access and resource manipulation.

Recommended validation test: Send 100 login attempts with wrong passwords in rapid succession. Assert the service returns 429 before attempt 100.

HIGH-06 — User PII Exposure: `GET /users/:id` is Unauthenticated and Returns

`passwordHash`

Affected files: `routes/user.routes.ts`, `controllers/user.controller.ts`,
`services/user.service.ts`

Exact scenario: `GET /users/:id` has no authentication middleware. `getUserById` returns the full Prisma `User` record, which includes `passwordHash`. The controller returns this directly via `createSuccessResponse(user)`.

Example attack/failure path:

- Attacker calls `GET /users/:id` for any user ID
- Response contains `passwordHash` (bcrypt hash)
- Combined with `GET /memberships/org/:id` (which includes full user objects), attacker can harvest password hashes for all members of all organizations

Business impact: Password hash exposure enables offline cracking attacks. PII leakage (email, name) violates GDPR and equivalent regulations.

Recommended validation test: Call `GET /users/:id` without auth. Assert 401. Call it with auth. Assert `passwordHash` is not present in the response body.

HIGH-07 — `GET /memberships/org/:id` Exposes All Member PII Without Authentication

Affected files: `routes/membership.routes.ts`, `controllers/membership.controller.ts`,
`services/membership.service.ts`

Exact scenario: `GET /memberships/org/:id` has no authentication middleware. `getOrganizationMembers` returns memberships with `include: { user: true }`, which includes the full `User` record (name, email, and — given HIGH-06 — `passwordHash`).

Example attack/failure path:

- Attacker iterates organization IDs (UUIDs, but GETable from the public `GET /organizations` endpoint)
- For each organization, calls `GET /memberships/org/:id` — receives full name, email, and role for every member
- Cross-references to build a complete directory of all platform users and their organizational roles

Business impact: Complete PII enumeration of the entire platform user base without any credentials. Enables targeted phishing, credential stuffing (HIGH-05), and social engineering.

Recommended validation test: Call `GET /memberships/org/:id` without authentication. Assert 401.

HIGH-08 — `withdrawOffer` Does Not Verify Ownership

Affected files: `services/resource-offer.service.ts` — `withdrawOffer()`,
`controllers/resource-offer.controller.ts` — `withdrawOfferController()`

Exact scenario: `withdrawOffer(offerId)` fetches the offer, checks status is `PENDING`, and sets it to `WITHDRAWN`. It does not check that the calling organization is `offer.offeringOrganizationId`. The controller is authenticated (`requireOrganizationRole`), but the organization context is not validated against the offer's owner.

Example attack/failure path:

- Org A creates an offer. Org B (or any authenticated org member) calls `POST /api/offers/:id/withdraw`
- The middleware verifies the caller has a role in their own organization, not Org A
- Org A's offer is withdrawn without their knowledge or consent

Business impact: Organizations can sabotage each other's offers, disrupting resource fulfillment.

Recommended validation test: As a user from Org B, attempt to withdraw an offer created by Org A. Assert 403.

HIGH-09 — No Pagination Cap: Unbounded `take` Parameter on List Endpoints

Affected files: All list controllers (`getTransfers`, `getResourceNeeds`, `getResourceOffers`, `getResourceLots`, `getAuditLogs`, etc.)

Exact scenario: All list endpoints accept a `limit` (or `take`) query parameter and pass it directly to Prisma's `findMany`. There is no maximum cap. A caller can pass `limit=1000000` and Prisma will attempt to load every matching record into memory.

Example attack/failure path:

- Attacker calls `GET /api/audit?take=1000000` with a valid OWNER token
- Database executes a full table scan of `AuditLog` for that organization
- Node.js process allocates memory for the entire result set
- OOM kill or connection pool exhaustion; other organizations' requests time out

Business impact: Denial of service via memory exhaustion. A single authenticated request per organization can take down the entire service.

Recommended validation test: Call any list endpoint with `limit=1000000`. Assert the response contains at most a capped number of records (e.g., 100) and the server remains responsive.

HIGH-10 — Transfer Audit Log Records Incorrect `previousStatus`

Affected files: `controllers/transfer.controller.ts` — `cancelTransferController()`

Exact scenario: The `cancelTransferController` creates an audit log entry hardcoding `previousStatus: 'PENDING'`. However, a transfer can be cancelled from either `PENDING` or `IN_TRANSIT` state (both are valid per the state machine). When an `IN_TRANSIT` transfer is cancelled, the audit log falsely records `previousStatus: 'PENDING'`.

Example attack/failure path:

- Transfer is started (moved to `IN_TRANSIT`)
- Transfer is then cancelled
- Audit log records `previousStatus: 'PENDING'` — investigators cannot reconstruct the true state sequence

Business impact: Audit log integrity is compromised. Forensic investigation of incidents is unreliable. Compliance failures for regulated industries.

Recommended validation test: Cancel an `IN_TRANSIT` transfer. Assert the audit log records `previousStatus: 'IN_TRANSIT'`.

MEDIUM SEVERITY

MED-01 — `safeAudit` Silently Swallows Audit Failures

Affected files: Referenced across all controllers; the `utils/safe-audit.ts` implementation (not in zip but implied by usage pattern)

Exact scenario: All controllers wrap `createAuditLog` calls in `safeAudit(() => ...)`. By design this catches and suppresses audit failures so the primary operation succeeds. This means critical business operations (offer acceptance, transfer completion, membership creation) can succeed with no audit record, and neither the caller nor any monitoring system is notified.

Example attack/failure path:

- Database connection pool is exhausted or `AuditLog` table is locked
- `acceptOffer` completes successfully, inventory is decremented, transfer is created
- Audit write fails silently — `safeAudit` swallows the error
- No record exists of who accepted the offer or when
- Repeated failures create permanent gaps in the audit trail

Business impact: Compliance violations (SOC 2, ISO 27001, GDPR audit requirements). Inability to investigate security incidents. The audit trail is untrustworthy under any load or failure condition.

Recommended validation test: Simulate an audit DB failure (e.g., kill the audit connection mid-request). Assert that either the primary operation rolls back, OR the failure is recorded in application logs with sufficient detail for reconstruction.

MED-02 — `cancelResourceNeed` Audit Log Hardcodes `previousStatus: 'OPEN'`

Affected files: `controllers/resource-need.controller.ts` — `cancelResourceNeedController()`

Exact scenario: The audit log for need cancellation hardcodes `previousStatus: 'OPEN'`. However, a need can be in `PARTIALLY_FULFILLED` status when cancelled (the service allows this). The audit log is factually wrong for this case.

Business impact: Same class of problem as HIGH-10 — audit trail inaccuracy undermines forensic investigation.

Recommended validation test: Cancel a `PARTIALLY_FULFILLED` need. Assert the audit log records the correct `previousStatus`.

MED-03 — Event and Lot Read Endpoints Expose Data Across Organizations Without Filtering

Affected files: `services/event.service.ts` — `getEvents()`, `routes/event.routes.ts`

Exact scenario: `GET /events` calls `prisma.event.findMany()` with no `where` clause. It returns all events from all organizations on the platform. There is no organization-scoped filtering unless the caller supplies `organizationId` as a query param — but this is optional and not enforced.

Example attack/failure path:

- Org A calls `GET /events` with no filters
- Receives all events from all organizations on the platform
- Can enumerate organizational activity, event timing, descriptions, and resource needs attached to events

Business impact: Cross-organization data leakage. Organizations can surveil each other's planned events and resource requirements.

Recommended validation test: Call `GET /events` without an `organizationId` filter. Assert only public events (if that concept exists) or only the caller's own org's events are returned.

MED-04 — Organization `status` Field is a Free String With No Enforcement

Affected files: `schema.prisma` — `Organization` model; `services/organization.service.ts`

Exact scenario: `Organization.status` is a plain `String` with a default of `"ACTIVE"`. There is no enum, no validation in the service layer, and no middleware checking whether an organization is `ACTIVE` before allowing operations on its behalf. A deactivated or suspended organization can continue creating resource needs, accepting offers, and completing transfers.

Business impact: Deactivated organizations remain fully operational. No ability to suspend bad actors or enforce platform terms of service.

Recommended validation test: Set an organization's status to `"SUSPENDED"`. Attempt to create a resource need on its behalf. Assert the operation is rejected.

MED-05 — Membership Role is a Free String With No Enum Validation

Affected files: `schema.prisma` — `Membership` model; `services/membership.service.ts`

Exact scenario: `Membership.role` is a plain `String`. The authorization middleware checks `allowedRoles.includes(membership.role)`. There is no server-side validation that the role value supplied during membership creation is one of the known roles (`OWNER`, `ADMIN`, `COORDINATOR`, `MEMBER`). An ADMIN creating a membership can assign an arbitrary role string.

Example attack/failure path:

- Attacker who is an ADMIN creates a membership with `role: "SUPERADMIN"`
- No endpoint currently checks for `SUPERADMIN` — but if any future endpoint is added with a typo or check for `"superadmin"` vs `"SUPERADMIN"`, behavior is unpredictable
- More practically: custom role strings may be used to confuse authorization checks

Business impact: Role enum pollution. Future development risk. Inconsistent authorization behavior.

Recommended validation test: Attempt to create a membership with `role: "GOD_MODE"`. Assert 400.

MED-06 — `updateResourceLot` Passes Raw `data` Object Directly to Prisma

Affected files: `services/resource-lot.service.ts` — `updateResourceLot()`

Exact scenario: After ownership and constraint validation, `updateResourceLot` calls `prisma.resourceLot.update({ where: { id }, data })` where `data` is the raw object passed in by the caller. There is no field allowlist. A caller could include `organizationId` in the update payload, changing ownership of a lot to another organization.

Example attack/failure path:

- Org A's ADMIN calls the update endpoint with `{ "quantity": 100, "organizationId": "org-B-id" }`
- Ownership check passes (Org A owns the lot at read time)
- The update writes `organizationId = org-B-id` — Org A has just transferred a ResourceLot to Org B without any authorization from Org B

Business impact: Unauthorized transfer of inventory ownership between organizations.

Recommended validation test: Update a lot while including `organizationId` in the payload pointing to a different org. Assert `organizationId` is unchanged after the update.

MED-07 — No Request Body Size Limit

Affected files: `app.ts`

Exact scenario: `app.use(express.json())` is called with no `limit` option. Express's default is 100kb. For `metadata` fields stored as `Json?` in `AuditLog` and for `notes` fields on various

models, an attacker can submit multi-megabyte payloads, consuming memory and database storage.

Business impact: Memory exhaustion and database bloat under adversarial input.

Recommended validation test: POST to any endpoint with a JSON body of 10MB. Assert the server returns 413 and does not persist the payload.

MED-08 — Integer Overflow Risk: `quantity` Fields Have No Upper Bound

Affected files: `schema.prisma` — all `Int` quantity fields; all service validation

Exact scenario: `ResourceLot.quantity`, `ResourceLot.availableQuantity`, `ResourceNeed.quantity`, `ResourceOffer.offeredQuantity`, `Transfer.quantity` are all `Int` (32-bit signed, max ~2.1 billion). No upper bound validation exists in service layer. PostgreSQL `Int` will overflow and go negative at 2,147,483,648. JavaScript's `parseInt` on query parameters has no bound either.

Business impact: Malicious quantity values can corrupt inventory arithmetic, produce negative available quantities, or cause database errors that propagate as 500s.

Recommended validation test: Submit `quantity: 2147483648` in a resource lot creation. Assert validation rejects it.

LOW SEVERITY

LOW-01 — Error Messages Leak Internal State

Affected files: `middleware/error.middleware.ts`

Exact scenario: The global error handler returns `err.message` directly in the response with no sanitization. Prisma throws detailed error messages that include table names, column names, constraint names, and query fragments. These are returned directly to the client.

Example attack/failure path:

- Attempting to create a duplicate resource returns `"Unique constraint failed on the fields: (name)"` — reveals column names
- Attempting an invalid foreign key returns the full Prisma error with relationship names

Business impact: Information leakage that assists schema reconnaissance. Facilitates more targeted injection or enumeration attacks.

Recommended validation test: Trigger a Prisma unique constraint violation. Assert the error response contains a generic message, not the Prisma error string.

LOW-02 — JWT `JWT_SECRET` Has Only `min(1)` Validation

Affected files: `env.ts`

Exact scenario: The Zod schema validates `JWT_SECRET` as `z.string().min(1)`. A one-character secret is accepted. If the secret is short, weak, or accidentally set to a placeholder like `"secret"` or `"changeme"`, all JWTs are trivially forgeable.

Business impact: If deployed with a weak secret (common in misconfigured environments), any attacker who guesses the secret can forge tokens for any `userId`.

Recommended validation test: Start the server with `JWT_SECRET=x`. Assert the server refuses to start with a clear error about secret strength.

LOW-03 — No HTTP Security Headers

Affected files: `app.ts`

Exact scenario: There is no `helmet` or equivalent middleware. The API responses lack `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, `Content-Security-Policy`, and `X-XSS-Protection` headers.

Business impact: Increased attack surface for clients consuming the API. If the API is ever accessed via browser (e.g., a frontend SPA), clickjacking and MIME-sniffing attacks are possible.

Recommended validation test: Make any request and assert the response includes at minimum `X-Content-Type-Options: nosniff` and `Strict-Transport-Security`.

LOW-04 — No CORS Configuration

Affected files: `app.ts`

Exact scenario: No `cors` middleware is configured. Express's default behavior does not add CORS headers, which may block legitimate browser clients — but more importantly, there is no explicit allowlist of permitted origins.

Business impact: Unpredictable cross-origin behavior. No ability to enforce origin policy when a frontend is added.

Recommended validation test: Send an `OPTIONS` preflight request. Assert the response contains appropriate `Access-Control-Allow-Origin` headers.

LOW-05 — Inconsistent Route Prefix Convention

Affected files: `app.ts`

Exact scenario: Some route groups are mounted at `/organizations`, `/users`, `/memberships`, `/partnerships`, `/events` (no `/api` prefix) while others are at `/api/resources`, `/api/resource-lots`, `/api/needs`, `/api/offers`, `/api/transfers`, `/api/audit`. This inconsistency will cause API gateway misconfiguration, incorrect rate limit scoping, and documentation errors.

Business impact: Operational risk. API gateway authentication bypass if WAF/proxy rules are applied only to `/api/*`.

Recommended validation test: Audit gateway/proxy rules. Ensure auth enforcement applies to all route groups, not just `/api/*`.

LOW-06 — Transaction Timeout Set to 30 Seconds

Affected files: `services/transfer.service.ts`, `services/resource-offer.service.ts`, `services/resource-need.service.ts`

Exact scenario: All critical transactions use `{ timeout: 30000 }`. The comment says this is to "match jest.setTimeout to prevent stale transaction errors" — meaning this was tuned for test ergonomics, not production performance. Under load, 30-second transactions will hold row locks, exhaust connection pools, and cascade into full service unavailability.

Business impact: Under production load, slow transactions hold locks for up to 30 seconds. With 10 concurrent transfers, the connection pool can be exhausted entirely.

Recommended validation test: Simulate 20 concurrent transfer completions with artificial delay. Assert connection pool is not exhausted and p99 latency remains under 2 seconds.

Summary Table

ID	Severity	Area	One-Line Description
CRIT-01	Critical	Inventory	Double-spend race on concurrent offer acceptance
CRIT-02	Critical	Inventory	TOCTOU in offer creation — no inventory lock
CRIT-03	Critical	Authorization	Transfer state changes have no party ownership check
CRIT-04	Critical	Authorization	ResourceLot/Need creation unauthenticated
CRIT-05	Critical	Authorization	ResourceOffer creation unauthenticated
CRIT-06	Critical	Authorization	Partnership endpoints entirely unprotected
CRIT-07	Critical	Authorization	Client-supplied userId used as transfer approver
CRIT-08	Critical	Concurrency	No pessimistic lock in acceptOffer transaction
HIGH-01	High	Authorization	Membership status (PENDING) not checked before granting access
HIGH-02	High	State Machine	Need cancellation ignores ACCEPTED offers and active transfers
HIGH-03	High	Concurrency	Transfer completion fulfillment calc races under concurrency

ID	Severity	Area	One-Line Description
HIGH-04	High	Concurrency	updateResourceLot TOCTOU — read/write not in transaction
HIGH-05	High	Security	No rate limiting on auth endpoints
HIGH-06	High	Security	Unauthenticated GET /users/:id returns passwordHash
HIGH-07	High	Security	Unauthenticated member PII enumeration
HIGH-08	High	Authorization	withdrawOffer has no ownership check
HIGH-09	High	Reliability	Unbounded pagination — no take/limit cap
HIGH-10	High	Audit	Transfer cancel audit log hardcodes wrong previousStatus
MED-01	Medium	Audit	safeAudit silently swallows all audit failures
MED-02	Medium	Audit	Need cancel audit log hardcodes wrong previousStatus
MED-03	Medium	Isolation	GET /events returns all orgs' events with no filtering
MED-04	Medium	State Machine	Organization status not enforced at operation time
MED-05	Medium	Validation	Membership role is a free string — no enum enforcement
MED-06	Medium	Security	updateResourceLot passes raw data to Prisma — ownership reassignable
MED-07	Medium	Reliability	No request body size limit
MED-08	Medium	Validation	No upper bound on Int quantity fields
LOW-01	Low	Security	Error messages leak internal Prisma state
LOW-02	Low	Security	JWT_SECRET has only min(1) validation
LOW-03	Low	Security	No HTTP security headers (helmet)
LOW-04	Low	Security	No CORS configuration
LOW-05	Low	Ops	Inconsistent route prefix convention
LOW-06	Low	Reliability	Transaction timeout of 30s tuned for tests, not production

Immediate Pre-Launch Blockers (Must Fix)

1. **CRIT-01 / CRIT-08:** Add pessimistic locking ((`SELECT ... FOR UPDATE`) via (`prisma.$queryRaw`) on (`ResourceLot`) rows inside all inventory-affecting transactions.
2. **CRIT-03:** Verify that the acting organization is a party to any transfer before allowing state transitions.
3. **CRIT-04 / CRIT-05:** Add (`authenticate`) + (`requireOrganizationRole`) to ResourceLot creation, ResourceNeed creation, and ResourceOffer creation.
4. **CRIT-06:** Add authentication and authorization to both partnership endpoints.
5. **CRIT-07:** Replace (`req.body.userId`) with (`req.user.id`) as (`approvedById`).
6. **HIGH-05:** Add rate limiting to (`/api/auth/login`) and (`/api/auth/register`).
7. **HIGH-06 / HIGH-07:** Strip (`passwordHash`) from all user responses; authenticate the membership list endpoint.
8. **HIGH-02:** Define and implement what happens to ACCEPTED offers and active transfers when a need is cancelled.
9. **HIGH-09:** Enforce a maximum value on (`limit`)/(`take`) across all list endpoints.